

임정규 / 백엔드 개발자

실패 이후까지 설계하는 백엔드 개발자

LG CNS 컨소시엄 전송형 전자영장 프로젝트에서 독립망 간 기관 연계를 개발하고 있습니다.
공공 시스템 운영 장애 대응과 개인 프로젝트의 먹등성, 데이터 정합성, 복구 경로를 함께
담았습니다.

BATON / GO

8 → 1

동시에 들어온 같은 링크 생성 요청을 1건으로
처리

happyGallery

220 / 218

OpenAPI operations / REST Docs tests

전송형 전자영장 시스템

전담

독립망 간 통신사실확인자료 개선과
집행포털 연계 담당

WORKING PRINCIPLES

정합성과 복구 경로를 먼저 설계합니다.

기능이 정상 동작하는 순간뿐 아니라 중복 요청, 외부 연동 실패, 재시작 뒤의 상태까지 한 흐름으로 봅니다.

01 경계를 먼저 정합니다 데이터 소유권, 트랜잭션 범위, 서비스별 책임을 먼저 나눕니다.	02 재처리를 기능으로 만듭니다 역등 키, 락, 아웃박스 등 복구 작업으로 실패 뒤의 동작을 정의합니다.	03 근거를 남깁니다 ADR, API 계약, 통합 테스트와 로그로 설계 이유와 결과를 확인합니다.
--	--	--

EDUCATION

2023.05 - 2023.11

카카오 클라우드 스쿨 개발자 과정 3기

6인 팀

WebRTC/HLS 현장강의 보조 서비스**2023.09.01 - 2023.11.10**

HLS 서버와 React 화면을 맡았습니다. WebSocket 제어와 WebRTC/RTP 미디어 경로를 분리하고 FFmpeg과 GStreamer로 HLS를 변환해 지연을 약 30초에서 11초로 줄였습니다.

CAREER

2026.03 - 진행 중
LG CNS 컨소시엄 참여 프로젝트**전송형 전자영장 시스템****해양경찰 KICS 연계 개발**

해양경찰 KICS 통신사실확인자료 개선과 집행포털 연계를 담당하며 독립망 사이의 요청과 제출 자료를 인터페이스와 Spring Batch로 연결하고 있습니다.

2024.06 - 2026.01
BEINTECH**차세대 군사법 정보 시스템****백엔드 개발 및 운영**

차세대 군사법 정보 시스템 군교정 부문에서 기관 연계 배치, 보안 기능, 로그 및 DB 기반 장애 대응을 담당했습니다.

Personal Activity

개발 서적 그룹 스터디 / 발표 및 Q&A 정리

LnS (Learn & Share) — HTTP 완벽 가이드

HTTP 메시지, 캐시, 프록시와 인증 등 실무에서 자주 확인하는 주제를 발표하고 질문과 답변을 문서로 정리했습니다.

[LnS 발표 및 Q&A 기록 ↗](#)

개발 서적 그룹 스터디 / 아이템별 학습 내용 기록

Effective Java 스터디

객체 생성, 불변성, 제네릭과 API 설계 원칙을 아이템별로 학습하고 적용 기준을 Notion에 기록했습니다.

[Effective Java 학습 기록 ↗](#)

CAREER PROJECT / LG CNS 컨소시엄

전송형 전자영장 시스템

LG CNS 컨소시엄 참여 프로젝트에서 사법기관 KICS, 전자영장 집행포털, 금융기관 및 통신사처럼 독립된 망 사이의 요청과 제출 자료를 연계하는 인터페이스와 Spring Batch를 개발하고 있습니다.

MY ROLE

해양경찰 KICS 통신사실확인자료 개선 및 독립망 간 전자영장 집행포털 연계

REQUEST

사법기관 KICS 독립망



INTEGRATION

전자영장 집행포털 인터페이스 및 Spring Batch



RESPONSE

금융기관 및 통신사 독립망

사법기관 KICS, 전자영장 집행포털, 금융기관 및 통신사가 서로 독립된 망에서 요청과 제출 자료를 주고받는 흐름을 공개 가능한 수준으로 단순화했습니다.

전담

해양경찰 KICS 연계

Cursor

대용량 상태 조회

Retry

콜백 순서 경합 복구

대표 문제 해결

CASE 01

신규 화면은 커서, 기존 화면은 지연 조인을 적용한다

문제 상황 전송 상태와 수신 자료가 계속 쌓이면 OFFSET이 커질수록 뒤쪽 페이지 조회 비용이 증가합니다.

해결 신규 전송 상태 조회에는 커서 페이지를 적용했습니다. 번호 이동이 필요한 기존 대용량 화면은 커버링 인덱스로 키를 먼저 찾고 본문을 지연 조인했으며, 데이터가 적은 화면에는 적용하지 않았습니다.

트레이드오프 커서 페이지는 임의 페이지 이동이 어렵고 지연 조인은 SQL이 복잡해집니다.

CASE 02

기관 연계 기능의 공통 처리 흐름을 재사용한다

문제 상황 수신 자료와 통신사실확인자료의 화면, 인터페이스와 Spring Batch 흐름이 비슷해 기능마다 같은 분기와 변환 코드를 만들 가능성이 컸습니다.

해결 공통 처리 흐름은 제네릭과 enum으로 묶고 조회, 변환과 전송은 책임별 클래스로 나눴습니다. 기관별 차이는 별도 구현으로 분리했습니다.

트레이드오프 공통 구조를 설계하는 동안 첫 기능의 개발 속도는 느려졌습니다.

CASE 03

먼저 도착한 PDF 콜백을 재조회로 복구한다

문제 상황 PDF 변환 콜백이 요청 상태 저장보다 먼저 도착하면 콜백 처리 시 조회 대상이 없어 정상 결과를 반영하지 못할 수 있었습니다.

해결 Spring Retry에 지수 백오프와 지터를 적용해 짧은 간격으로 상태를 다시 조회했습니다. 동시에 몰린 콜백의 재시도 시점도 분산했습니다.

트레이드오프 재시도는 정해진 횟수 안에서만 수행합니다.

CASE 04

외부 API 호출과 DB 트랜잭션을 분리한다

문제 상황 주기적으로 들어오는 연계 요청이 겹치고 외부 승인 API 응답이 늦어지면 같은 작업이 중복 실행되거나 DB 연결을 오래 점유할 수 있었습니다.

해결 외부 호출 전후의 DB 반영을 REQUIRES_NEW 트랜잭션으로 분리하고, 단일 애플리케이션 안에서는 ReentrantLock으로 겹친 실행을 막았습니다.

트레이드오프 ReentrantLock은 한 JVM 안에서만 유효합니다.

공개 범위

LG CNS 컨소시엄 참여 프로젝트로 현재 진행 중인 공공 SI입니다. 독립망 간 연계 구조와 직접 수행한 역할은 공개하고, 실제 접속 주소, 운영 값, 보안 설정, 소스 코드와 내부 문서만 제외했습니다.

CAREER PROJECT / PUBLIC SI

차세대 군사법 정보 시스템

폐쇄망과 레거시 환경에서 기관 연계 배치와 보안 기능을 개발하고 운영 장애를 분석했습니다.

**JENKINS****기관 연계 배치 3종**

실행 결과 → 서버 로그 → DB 반영 → 화면
조회

**BUSINESS SYSTEM****군교정 업무**

검증 · 반영 · 운영 대응

3종

기관 연계 배치

CSRF

요청 위조 방지

Tika

파일 형식 검사

개발**기관마다 다른 데이터 연계를 배치로 처리한다**

연계 배치 3종을 기관별 입력 조건과 처리 순서에 맞춰 구현하고 Jenkins에서 실행과 재처리를 관리했습니다.

보안**기존 화면에 CSRF 방어와 파일 형식 검사를 추가한다**

Spring Security의 CSRF 토큰을 기존 화면에 연결하고 Apache Tika로 파일의 실제 형식을 확인했습니다.

장애 대응**화면 오류를 로그, DB, 배치 순서로 추적한다**

SSH 서버 로그, DB 데이터, 배치 상태를 같은 시간 순서로 비교하고 필요하면 기관 담당자와 연계 시간을 확인했습니다.

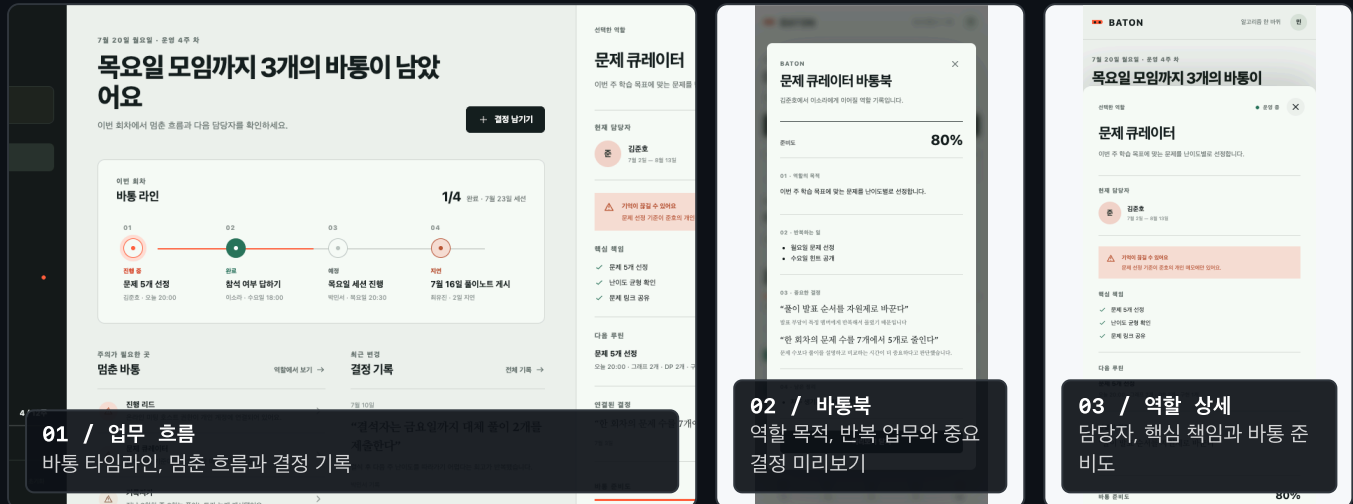
공개 범위**직접 수행한 업무만 비식별화**

보안이 필요한 공공 프로젝트이므로 기관과 데이터는 공개하지 않고 직접 수행한 개발과 운영 업무만 정리했습니다.

PERSONAL PROJECT / MAIN PROJECT

BATON

조직 운영의 기준 데이터는 Core에 두고, 링크, URL 점검, 메시지 전송을 실패 특성에 따라 별도 마이크로서비스로 분리했습니다.



서비스 구성

CORE / SYSTEM OF RECORD
조직 운영 기준 데이터
팀, 시즌, 역할, 루틴, 라운드, 의사결정, 자료, 핸드오프

MySQL · PRD 5 · ADR 17 · OpenAPI 계약

GO / MICROSERVICE
정책 기반 링크
UUID 역등 키, HMAC-SHA256 코드, 허용 경로만 처리

MySQL · 374 tests · 동시 8 → 1

WATCH / MICROSERVICE
URL 상태 점검
DNS pinning, SSRF 방어, lease 및 revision 펜싱, 아웃박스

PostgreSQL · 354 tests

RELAY / MICROSERVICE
메시지 전달
inbox 중복 제거, 작업 선정, 재시도, 처리 결과 미확인 상태

PostgreSQL · 373 tests

현재 상태

각 서비스의 핵심 기능과 테스트는 구현했습니다. 서비스 간 전체 연동과 운영 환경 배포는 진행 중입니다. 문서와 테스트 수치는 2026.08.09 저장소 기준입니다.

PERSONAL PROJECT / BATON EVIDENCE

대표 문제 해결

서비스마다 실제로 다루는 실패 단위와 복구 기준을 문서와 테스트로 고정했습니다.

CORE / HANDOFF**역할 교대의 중간 상태를 한 트랜잭션으로 묶는다**

문제 상황	수락과 담당자 변경이 따로 반영되면 책임자가 섞일 수 있음
해결	PREPARING → TRANSFERRED → ACCEPTED와 열린 바통 1건 제약
트레이드오프	상태와 이력이 늘어 운영 화면과 복구 절차가 필요

GO / IDEMPOTENCY**GO의 링크 생성은 재시도해도 같은 결과를 돌려준다**

문제 상황	응답 유실과 동시 요청으로 같은 링크가 중복 생성될 수 있음
해결	UUID 먹등 키, 요청 해시와 HMAC 기반 고정 링크 코드
트레이드오프	HMAC 키와 DB를 같은 시점에 복구해야 함

WATCH / SAFE CHECK**느린 URL 점검 중 DB 락을 잡지 않는다**

문제 상황	느린 I/O와 늦은 결과가 경합 및 최신 상태 덮어쓰기를 만들
해결	SSRF 차단, DNS pinning, lease와 revision 펜싱
트레이드오프	실제 지연 분포에 맞춰 lease 시간을 계속 조정해야 함

RELAY / RECOVERY**전송 결과를 모르면 원본 기록을 바꾸지 않는다**

문제 상황	응답 유실 뒤 재시도가 중복 발송으로 이어질 수 있음
해결	불변 전송 시도 기록, 전송업체용 먹등 키와 OUTCOME_UNKNOWN
트레이드오프	자동 재전송 대신 결과 조회 및 운영자 조정 절차가 필요

PERSONAL PROJECT / BATON DOCUMENTS

문서 분류와 대표 문서

기술 선택, 복구 절차와 서비스 계약을 다음 변경 때 확인할 수 있는 기록으로 남겼습니다.

문서 분류

2026.08.09 저장소 기준

PRD 14 서비스별 책임, 입력 계약과 완료 조건을 정의합니다.	ADR 43 기술 선택의 이유, 대안과 트레이드오프를 기록합니다.	Runbook 5 배포, 복구와 공개 스테이징 환경의 검증 절차를 정리합니다.	API Contract 1 Core OpenAPI를 서비스 계약의 기준으로 관리합니다.
--	--	--	--

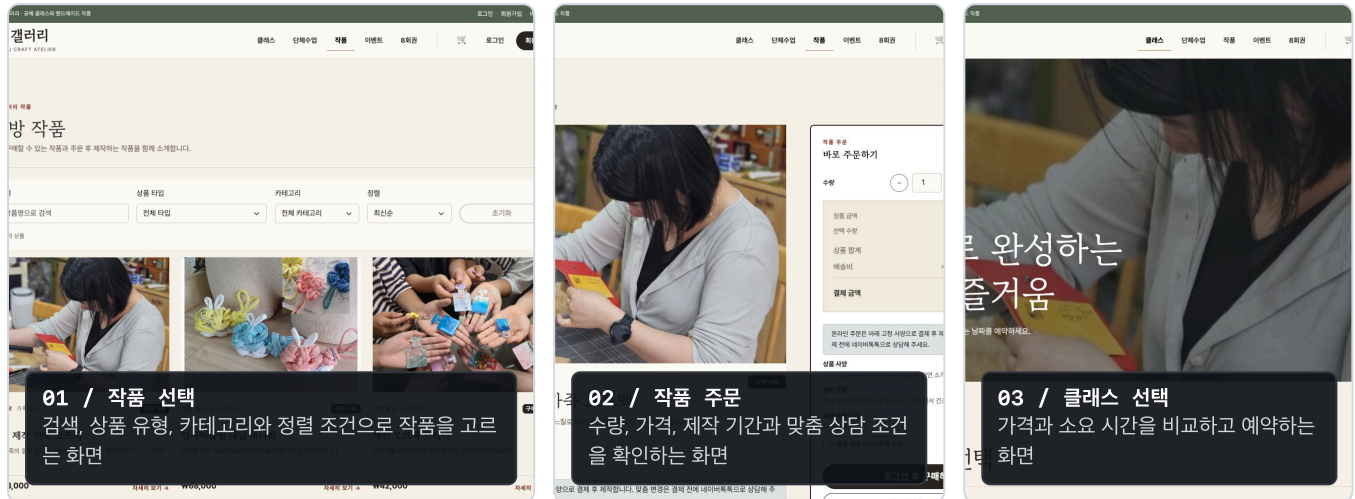
대표 문서

[ADR 요약] Core 핵사고날 아키텍처 비공개 원문에서 공개 가능한 결정과 트레이드오프를 요약	[ADR 요약] GO 먹등 링크 생성 동시 요청과 재시도에도 링크를 한 건만 생성하는 방식
[ADR] WATCH 상태 변경 이벤트 전달 아웃박스 기반 상태 변경 이벤트 전달 원문	[Runbook] WATCH 공개 스테이징 전달 검증 최초 전달, 응답 유실 재전송과 적체 이벤트 복구를 검증하는 운영 절차
[ADR 요약] RELAY 전송 시도 복구 외부 호출 전 시도 의도를 저장하고 복구하는 결정 요약	

PERSONAL PROJECT / COMMERCE + BOOKING

happyGallery

결제와 환불 결과를 확인할 수 없는 상태, 알림 프로세스 중단, 예약 및 재고 경쟁을 복구 가능한 상태로 저장했습니다.



AWS

운영 환경 가동 후 종료

220

OpenAPI operations

218

REST Docs 테스트

구조

의존 방향을 bootstrap → web, persistence, external 어댑터 → application → domain 순서로 제한했습니다. 모든 클래스에 인터페이스를 만들지 않고 결제와 알림처럼 교체 가능한 외부 연동에만 포트를 뒀습니다.

모듈과 타입 수는 늘지만 의존 위반을 빌드에서 차단할 수 있습니다. 현재 규모에서는 domain 모듈에 일부 JPA 매핑 어노테이션을 유지해 분리 비용을 줄였습니다.

운영 상태

AWS에 운영 배포했으나 트래픽과 무관한 상시 리소스 비용이 발생해 운영을 종료했습니다. 현재 공개 URL은 없으며 API와 테스트 수치는 2026.08.09 로컬 저장소 기준입니다.

PERSONAL PROJECT / HAPPYGALLERY EVIDENCE

대표 문제 해결

정합성, 외부 I/O, 보안과 운영 비용을 각각 검증 가능한 경계로 나눴습니다.

ARCHITECTURE

계층 규칙은 코드 리뷰가 아니라 빌드에서 막는다

문제 상황 web과 persistence 코드가 도메인으로 섞이기 쉬움

해결 6개 모듈, Gradle 의존성과 ArchUnit 정책 테스트

트레이드오프 타입 수는 늘고 domain의 일부 JPA 의존은 유지

PAYMENT / REFUND

결제와 환불 결과를 모를 때 같은 작업을 무작정 반복하지 않는다

문제 상황 PG 응답 유실 뒤 중복 승인과 환불 위험

해결 멍등 키, processingToken, 결과 조회 후 재시도

트레이드오프 API 응답 시 REQUESTED 상태가 남을 수 있음

NOTIFICATION

업무는 저장됐지만 알림 작업이 사라지는 틈을 없앤다

문제 상황 업무 커밋 직후 종료되면 알림 요청이 사라짐

해결 같은 트랜잭션 아웃박스, 즉시 전송과 스케줄러 복구

트레이드오프 비동기 지연과 at-least-once 중복 가능성

BOOKING / STOCK

예약과 재고의 락 순서를 고정한다

문제 상황 마지막 좌석과 재고가 동시 요청에서 초과 처리될 수 있음

해결 비관적 락과 클래스→슬롯, productId 고정 순서

트레이드오프 같은 행에 요청이 몰리면 대기 시간이 증가

PERSONAL DATA

개인정보 복원과 검색의 키를 분리한다

문제 상황 평문 제거와 주문 정확 검색을 함께 지원해야 함

해결 AES-GCM 암호화와 HMAC 블라인드 인덱스

트레이드오프 부분 검색 미지원, 키 유실 방지 절차 필요

OPERATIONS / COST

AWS 고정비를 확인하고 운영 구조를 다시 선택한다

문제 상황 트래픽과 무관한 상시 리소스 비용 발생

해결 비용 원인 확인, 리소스 종료, 단일 노트북 k3s 준비

트레이드오프 비용은 줄지만 단일 노드는 고가용성을 제공하지 않음

PERSONAL PROJECT / HAPPYGALLERY DOCUMENTS

문서 분류와 대표 문서

요구사항, 설계 결정과 운영 회고를 구현 및 변경의 기준으로 관리했습니다.

문서 분류

2026.08.09 저장소 기준

PRD 4 상품, 예약, 주문과 운영 정책의 기준을 관리합니다.	ADR 44 아키텍처, 동시성, 결제와 보안 결정을 기록합니다.	Idea / POC 39 / 1 구현 전 가설과 외부 장애 대응 방식을 검증합니다.
Retrospective 10 운영 비용, 테스트 흐름과 실패 원인을 되짚습니다.	Runbook 1 k3s 배포, 롤백, 백업과 복구 절차를 관리합니다.	

대표 문서

[PRD] 제품 기준 스펙 상품, 예약, 주문과 운영 정책의 상위 기준을 정한 문서	[ADR] hexagon 아키텍처 전환 6개 모듈의 의존 방향과 포트 적용 범위를 정한 기록
[ADR] 알림 Outbox 전달 보장 같은 트랜잭션 저장과 커밋 이후 복구 방식을 정한 기록	[ADR] 개인정보 암호화와 블라인드 인덱스 복원은 AES-GCM, 정확 검색은 HMAC으로 분리하고 키 회전 범위를 정한 기록
[Retrospective] AWS 비용과 운영 종료 상시 리소스 비용을 확인하고 운영 환경을 내린 과정	

STACK WITH CONTEXT

사용한 기술과 적용 경험

기술 이름보다 어떤 문제에 적용했고, 실패 뒤 어떻게 복구했는지를 함께 설명합니다.

Backend

Java 21 / 11 / 8, Spring Boot, Spring Batch, eGov 4.1, JPA, MyBatis

Architecture

헥사고날 아키텍처, 모놀리식 애플리케이션, Gradle 멀티모듈, 마이크로서비스

Data & Recovery

MySQL, PostgreSQL, Tiberio, Redis, 멍든 처리, 아웃박스, 보상 처리

Test & Operations

JUnit, Testcontainers, REST Docs, Playwright, Docker, Jenkins, Prometheus

요구사항, 장애 영향과 복구 방법을 확인한 뒤 필요한 구조를 선택합니다.

jolri24@naver.com

<https://github.com/ljkhyeong>

<https://ljkportfolio.netlify.app>

010 3972 6284

BATON Core

GO

WATCH

RELAY

happyGallery

전송형 전자영장

군사법 경력 사례

WebRTC/HLS